Practical and Efficient PUF-Based Protocol for Authentication and Key Agreement in IoT

Sivappriya Manivannan, Rajat Subhra Chakraborty, *Senior Member, IEEE*, Indrajit Chakrabarti, *Member, IEEE*, and Jothi Ramalingam, *Member, IEEE*

***Abstract*—The immense potential of the Internet of Things (IoT) is challenged by grave security vulnerabilities that are easily exploitable in resource-constrained environments. We propose a lightweight Authentication and Key Agreement (AKA) protocol to derive a shared session key for each communicating node in a mutually communicating cluster of IoT nodes. Each IoT device is embedded with a Physically Unclonable Function (PUF), and a Fuzzy Extractor (FE) is deployed to correct and reproduce the private key and public helper data pair from the possibly erroneous PUF response. This secret raw PUF response is not stored explicitly in the server. A forward-secure authenticated key agreement is achieved by incorporating Elliptic Curve Diffie–Hellman (ECDH) key exchange protocol. The security of the proposed scheme has been formally verified while considering both active and passive attackers using the *Verifpal* tool. A prototype implementation with the arbiter PUF circuit, FE, and associated software has successfully demonstrated the efficacy of our scheme.**

***Index Terms*—Formal verification, fuzzy extractor (FE), Internet of Things (IoT), physically unclonable function (PUF).**

I. Introduction

THE Internet of Things (IoT) is a cyber–physical system that commonly includes sensors, actuators, and RFIDs. It allows the agents to autonomously process and exchange data over a communication network. At the same time, IoT devices face severe threats from attacks involving unauthorized access, data/identity theft, electronic counterfeiting, reverse engineering, and physical attacks. Given that IoT devices are often of small form factor and severely constrained with respect to processing power, memory, and energy, it is infeasible to execute computationally intensive traditional cryptographic algorithms. Hence, in the recent past, Physically Unclonable Function (PUF) has been explored as a hardware "root of trust" [1]. Ideally, each PUF instance can be associated with a unique, perfectly stable (i.e., reliable), and unclonable mapping between input stimuli ("challenges") and corresponding outputs ("responses"), which act as an electronic "fingerprint" of the device in which that particular PUF instance is embedded.

Manuscript received 11 June 2023; revised 2 July 2023 and 11 July 2023; accepted 24 July 2023. Date of publication 2 August 2023; date of current version 30 May 2024. This manuscript was recommended for publication by R. Salvador. *(Corresponding author: Sivappriya Manivannan.)*

Sivappriya Manivannan and Indrajit Chakrabarti are with the Department of Electronics and Electrical Communication Engineering, Indian Institute of Technology Kharagpur, Kharagpur 721302, India (e-mail: msivappriya@kgpian.iitkgp.ac.in; indrajit@ece.iitkgp.ac.in).

Rajat Subhra Chakraborty is with the Department of Computer Science and Engineering, Indian Institute of Technology Kharagpur, Kharagpur 721302, India (e-mail: rschakraborty@cse.iitkgp.ac.in).

Jothi Ramalingam is with the Department of Mathematical and Computational Sciences, National Institute of Technology Karnataka, Mangaluru 575025, India (e-mail: jothiram@nitk.edu.in).

Digital Object Identifier 10.1109/LES.2023.3299200

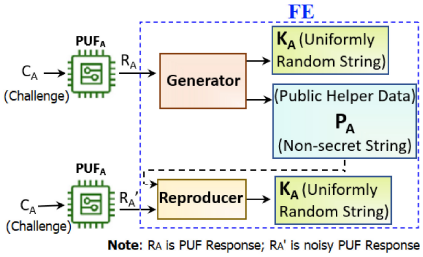

Fig. 1. FE for PUF [4].

Also, IoT device authentication is a problem of fundamental importance to enable secure and trustable communication. Storage of private keys in situ at the devices (e.g., in a nonvolatile memory module) are not secure due to the threat of physical attacks, while the option of storing the private key of *every* device at a central master authentication server is not scalable due to the sheer number of IoT devices, and the need to frequently update the key database due to the ever-growing and dynamic nature of IoTs. In the context of PUFs, in papers [2], [3], the challenge-response pair (CRP) database can be thought to be the identifier of the particular PUF instance. However, since the size of each of these CRP databases can run into thousands of CRPs, the storage of CRP databases of all IoT devices at a central server, or even over a distributed storage, is not practical. In contrast, our proposed protocol stores only a single processed CRP and does not use the verifier (central server) for secure communication. Moreover, the authors in the above-cited publications implemented Elliptic Curve Diffie–Hellman (ECDH) key exchange protocol with pairing, which is a computationally expensive operation. In our protocol, pairing has been avoided considering the computational resources needed for implementing Pairing Based Cryptography (PBC) without any compromise in security, to reduce resource footprint. Additionally, we have verified our protocol using a widely used formal security verifier software (*Verifpal*).

One of the major challenges of deploying PUFs in practice is that of imperfect reliability, whereby the response of the PUF to the same challenge could vary with changes in environmental conditions and age. The Fuzzy Extractor (FE) protocol [4] is a popular scheme based on error correction codes that is extremely effective in correcting noisy responses, as depicted in Fig. 1. The helper data can be easily stored in nonvolatile memory modules available on the resource-constrained IoT device and its public availability does not affect the security of the proposed scheme.

In this letter, we propose and formally prove the security of a novel PUF-based authenticated key exchange protocol, which is suitable for resource-constrained devices, such as IoTs. The proposed protocol stores FE-generated public helper data, and only one randomly generated private key pair from a CRP per

TABLE I
PROPOSED PROTOCOL NOTATIONS AND DEFINITIONS

Notations	Definitions
ID_A, ID_B	Identity of IoT device nodes A and B respectively
Reg_Req	Fixed "Registration Request" string
C_{A_1}, C_{B_1}	Random Challenge to node A and B respectively
R_{A_1}, R_{B_1}	Respective Response for challenges C_A and C_B
K_{A_1}, K_{B_1}	Generated and reproduced FE random priv. keys w.r.t. R_{A_1}, R_{B_1} respectively
P_{A_1}, P_{B_1},	Generated FE Helper Data w.r.t. R_{A_1} and R_{B_1} respectively
TS_{A_1}, TS_{A_2}, TS_{A_3}	Three timestamps on IoT node A
TS_{B_1}, TS_{B_2}, TS_{B_3}	Three timestamps on IoT node B
R'_{A_1}, R'_{B_1}	Error response for given challenges
C_{A_2}, C_{B_2}	New challenges after successful communication (for update)
R_{A_2}, R_{B_2}	Corresponding new responses after successful communication (for update)
K_{A_2} , K_{B_2}	Generated and reproduced FE random priv. keys w.r.t. R_{A_2}, R_{B_2} respectively
P_{A_2}, P_{B_2}	Generated FE Helper Data w.r.t. R_{A_2} and R_{B_2} respectively
t, s	Private keys for Elliptic Curve point multiplication
g	Generator point for Elliptic Curve

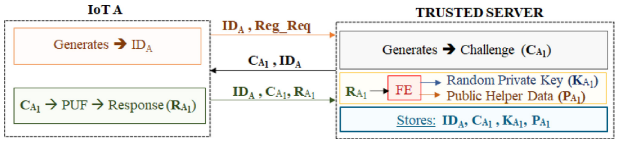

Fig. 2. Enrolment phase of the proposed scheme.

PUF instance on the authentication server, instead of a large CRP database. In case of any threat of security breach, a new CRP and the keys corresponding to it can be generated, and this relatively limited information can be updated easily on the server. Also, the scheme uses only two CRPs for every communication session between a pair of IoT devices. We successfully demonstrate the efficacy of the proposed protocol through a Field Programmable Gate Array (FPGA)-based prototype implementation using Arbiter PUF (APUF).

II. PROPOSED SCHEME

A detailed description of the proposed scheme for secure communication between two nodes ("IoT A" and "IoT B") is given, each of which has an integrated strong PUF and is registered through a trusted server. The scheme has four main components: 1) a strong PUF that is embedded in each IoT device for authentication; 2) a cryptographic hash function; 3) a FE to enhance the security; and 4) ECDH key exchange protocol. The proposed protocol has four main phases: 1) the enrolment phase; 2) mutual authentication phase; 3) authenticated key agreement phase; and 4) updated key establishment phase. These phases have been explained in detail next.

The notations used and their corresponding definitions are expounded in Table I.

A. Enrolment Phase

The PUF enrolment is a standard, one-time (prior to deployment) and *secure* phase with no opportunity to "spy" for an adversary. In case of an upgrade to the trusted server, the data can be backed up similarly to other secure databases. Since the PUF enrollment data is quite small, there is minimal storage overhead. The proposed protocol and the steps of this phase have been depicted in Fig. 2.

The IoT device sends the trusted server its ID with a request for registration. Upon receiving the request, the server sends a random challenge to the IoT with the device ID for verification. Consequently, the PUF embedded in IoT produces the response to the challenge received and replies with its ID and the obtained CRP. The server executes the FE on the acquired response to generate the random private key and public helper data. Finally, the server stores the credentials, viz., ID_A, C_{A_1}, K_{A_1}, and P_{A_1}.

After successfully registering the IoT devices with the trusted server, communication between any two nodes can be initiated through the server.

B. Mutual Authentication Phase

In the proposed mutual authentication phase, *only one pair of CRP is used instead of a set of CRPs*. The server can establish a secure authentication of the device as the challenge and corresponding credentials have already been stored during the registration phase for the device. Also, none of the potential information like the secret key or any public data is stored in the IoT device to overcome the physical attack. Fig. 3 details the information exchanged during the mutual authentication phase between two nodes through a server that involves the following three steps.

Step 1 - Identification Stage: PUF-based authentication is performed by comparing the generated response with that stored in the server for the same challenge generated during the enrolment phase.

Step 2 - Computational Stage: We consider the fact that the regenerated responses (R'_{A_1} and R'_{B_1}) for the same applied challenges (C_{A_1} and C_{B_1}) will be noisy due to changes in the environmental condition. This issue is overcome using the FE scheme. The computed credentials [HASH (C_{A_1}, TS_{A_2}, K_{A_1}) and HASH (C_{B_1}, TS_{B_2}, K_{B_1})] are compared with those received from the trusted server (TAG_{A_2} and TAG_{B_2}). If both match, then the authentication of two IoT devices with the server is successful.

Step 3 - Upgradation Stage: The new CRP is generated by the IoT device, along with the corresponding FE-generated random private key and public helper data, which is stored in the server for updating these credentials after a successful communication. The new challenge of IoT-A (C_{A_2}) is the concatenated (CONCAT) hash value of the previous challenge (C_{A_1}) and random private key (K_{A_1}). The corresponding response R_{A_2} is generated from the deployed PUF in the device when challenged with C_{A_2}. This response is input to the FE module to generate a random private key (K_{A_2}) and public helper data (P_{A_2}). Finally, the new random key is encrypted with the previous key and sent to the server along with the hash of a new timestamp (TS_{A_3}) and the keys, K_{A_1} and K_{A_2}. After this step, the server can verify and store the new key K_{A_2}, while performing decryption with its corresponding challenge (C_{A_2}).

C. Authenticated Key Agreement Phase

Authenticated Key Agreement (AKA) is a distinctive phase in the proposed scheme. A secure key exchange process between two IoT devices with an authenticated and agreed session key computation through the trusted server is the primary intention in this phase. It is achieved by incorporating the ECDH key exchange protocol as detailed in Fig. 4. The key aim is to exchange the keys g^t and g^s between two IoT devices and compute a newly agreed common secret session key.

Finally, IoT A and IoT B both have master keys g^{ts} and g^{st}, respectively, unknown to the server. To agree with the master keys, IoT A encrypts a random message (msg_A) with g^{ts} and sends the ciphertext (C) to IoT B through the server. IoT B decrypts the ciphertext using g^{st} to retrieve the message. Successful decryption establishes an agreement between the master keys. IoT B communicates the agreement by sending the encrypted message back to IoT A. A match with the message indicates the success of AKA phase.

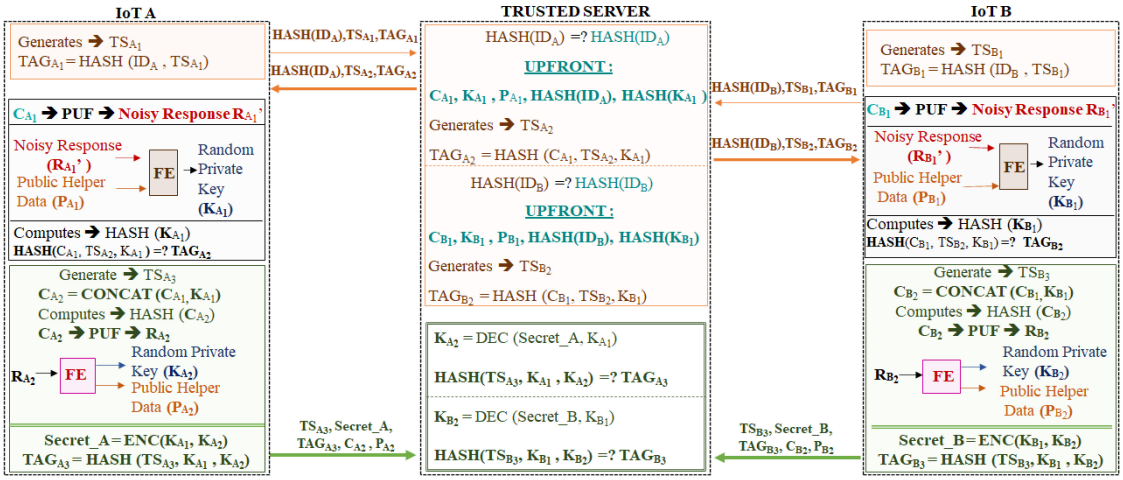

Fig. 3. Mutual authentication phase of the proposed scheme.

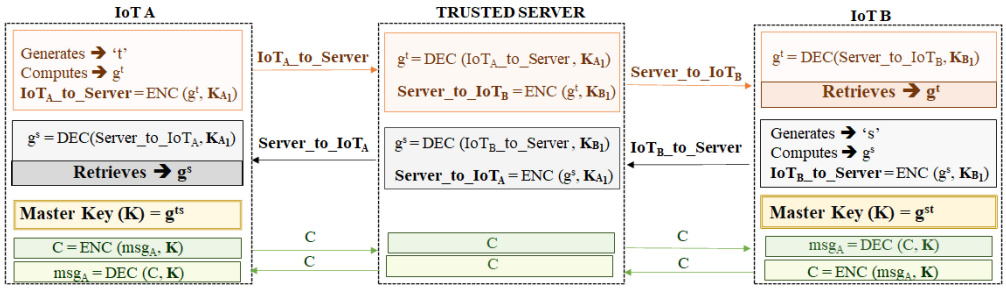

Fig. 4. Authenticated key agreement phase.

D. Updated Key Establishment Phase

As the session key has been computed and agreed upon during the authenticated key agreement phase, any message can be communicated between IoT A and IoT B, even without the server's interference in a highly secure way. In this phase, the new credentials with respect to IoT A (C_{A_2}, K_{A_2}, and P_{A_2}) and IoT B (C_{B_2}, K_{B_2}, and P_{B_2}) are updated, and the old credentials, viz., [C_{A_1}, K_{A_1}, and P_{A_1}] and [C_{B_1}, K_{B_1} and P_{B_1}] of respective IoTs are erased from the server.

III. Security Analysis

As *security assumption* for the proposed network model, we assume that the IoT devices are untrusted, as they are installed in a public (insecure) place without physical protection. In such cases, an adversary can perform any malicious action on the device, such as leakage of information from the local memory, eavesdropping, altering the messages, and replying to the messages. In addition, the malicious user will not have any access to the server's memory, as we consider the server to be trusted.

A. Cryptographic Attacks

Cryptographic attacks are attack models in cryptanalysis where the malicious system is fully or partially broken. Detailed attack scenarios to which the proposed protocol is resilient are explained in Table II.

B. Effects of Key Compromise

A compromise of a key's utilization leads to malicious usage of the secret key by illicit users, which results in compromised secure communication. The ways to achieve confidentiality in security are explained in Table III.

C. Formal Verification of the Proposed Scheme

Formal verification is an evidential correctness check of an intended protocol concerning specific formal properties. Many existing symbolic security analysis tools are available that aid in developing and applying formal, machine checkability approaches to analyze, design, and implement cryptographic concepts [6]. We have employed a recent symbolic verification tool *Verifpal*, which is considered a superior tool amongst peers [7], as it provides a highly intuitive framework for real-world protocol modeling and analysis via built-in primitive definitions. It also assists with queries for advanced security properties, such as key compromise impersonation and forward secrecy. *Verifpal* considers two varied adversarial models: 1) *unauthenticated-link adversarial model* (UM), where an active attacker exploits the network by altering the information and play-acts to vigorous attacks and 2) *authenticated-link adversarial model* (AM), where the passive attacker eavesdrops on the information being communicated, which stays unchanged throughout the attack. The *Verifpal* addresses the queries like confidentiality, mutual authentication, unlinkability, and freshness to analyze the contradictions after each run, and terminates when the attacker does not learn the values. Fig. 5 shows the result of verifying the proposed authenticated key agreement protocol using *Verifpal*, where all the queries are passed considering an active attacker.

IV. Prototype Implementation

We have implemented a prototype using a hardware-software co-design approach [2], in which a 64-bit APUF, a widely used delay PUF [8], on an *Xilinx Artix-7* series FPGA *Xilinx ISE Design Suite* (v. 14.7) was used to implement the APUF, with the controlled placement of the logic blocks enabled by custom scripts. Communication with the FPGA board was enabled.

TABLE II
CRYPTOGRAPHIC ATTACKS AND SECURITY SOLUTIONS

Attacks	A brief on attacks and resilience of proposed protocol to those attacks
Denial of Service (DoS)	The cyberattack floods the target network with traffic until a crash or access denial for legitimate users by generating valid requests. The responder does not take longer to respond because heavy point multiplication is not performed. Hence DoS attack fails.
Man-in-the-middle (MitM)	The active attacker manipulates the user to initiate a meeting between two parties and manipulates them to achieve his desired act of getting access to the data that two parties decide to communicate with each other. As the encryption is carried out in every progressive step, the attacker fails to intrude and manipulate the server by providing falsified statements.
Attack on Two-Time Pad	The typical statute of cryptography is never to use the same secret key more than once, as the ciphertext intends to be vulnerable to ciphertext-only attacks. We use different keys for the encryption by updating the server with a new CRP and its corresponding secret keys after every successful communication where the server deletes all the details corresponding to the previously used key.
Replay Attack	The cybercriminal eavesdrops on the secure communication network and seizes private messages. The attacker fraudulently replays the previously observed messages to masquerade as the legitimate party to the verifier and vice versa. We change the session key for every valid communication. Also, we use timestamps on all the messages, which prevents the attacker from resending messages that take longer than the fixed time period.
Modeling Attack on APUF	An attacker with access to a small database of CRPs can build an accurate mathematical model of the APUF through machine learning, thereby enabling correct prediction of the PUF's responses to any arbitrary challenge with high probability of success. This attack can completely invalidate the security of any PUF based protocol. In our case, only one challenge is used to enroll the PUF instead of a large number of CRPs. We store the challenge and its corresponding FE-generated helper data and random private key in place of raw PUF response. To prevent an arbitrary adversary from querying the APUF on the deployed IoT device in-field, a one-time programmable fuse mechanism can be used where the PUF is queried only during enrolment from outside the device by Original Equipment Manufacturer (OEM). Once the PUFs' responses have been noted, the fuse is blown off before the device is deployed in the field, after which the PUF subsystem at the IoT node generates a new challenge internally. In fact, prior to the wide deployment of PUFs, this was a common technique to program device-specific secrets inside devices prior to deployment, and is still quite common in industrial products like System-on-chip from leading vendors, e.g., see [9].

TABLE III
WAYS TO ACHIEVE CONFIDENTIALITY IN SECURITY

Perfect Forward Secrecy (PFS)	PFS, also known as *Forward Secrecy (FS)*, is a featured key agreement protocol that assures no compromise in the previous session keys even if the long-term session key used in the communication is conceded. In our key agreement protocol, after every successful communication, a new session key is generated, so the data is not compromised.
Key Freshness	Key freshness is a presumable generation of the new session key in due time with the intent that no future interaction is withheld despite old keys being compromised. Freshness can be achieved by using timestamps, nonce, and key agreement protocol.
Key Erasure	Key erasure is a method of expunging the previous session key to protect past communications, although the current memory where the new session key is stored is compromised. In our designed protocol, the new session key is not compromised as even the server will not know the session key and is not stored elsewhere.

Fig. 5. Successful execution of proposed protocol on Verifpal.

The FE for PUF reliability improvement was implemented in hardware following the efficient scheme introduced in [12], and in software (for server) using the *python-fuzzy-extractor* utility [11]. The hardware required for a 64 output-bit FE decoder was 488 slices (about 0.7% of available slices on the FPGA), 6495 clock cycles, supported 100-MHz clock speed, at a total of 1.6 ms (approximately) for the key and helper data generation (these results are in excellent agreement with [12]). The overall FE will require less than 7% of the FPGA resources. The total execution time for the software FE was about 0.72 s on a Linux workstation with a 2.70-GHz processor and 8-GB RAM, of which the generator and reproducer modules take about 0.50 and 0.22 s, respectively.

To derive a 256-bit response starting with a 256-bit challenge, the LFSR loads an input seed (S_i) of 256 bits for a particular challenge (C_i). A 64-bit bitstream requires 64 clock cycles and constitutes an input challenge to 16 APUFs in parallel, which together produce a 16-bit response in one clock cycle. The same procedure is repeated 16 times (16×16) to get a 256-bit response. The total number of clock cycles used collectively for LFSR ($64 \times 16 = 1024$ clock cycles) and 16 APUFs in parallel to obtain 256-bit CRPs is 1040.

We have executed ECDH based on Koblitz curve (*secp256k1*) to generate the agreed session key from *Asecuritysite* [10], and the *hashlib* software library for SHA3-256 hash function to map a point on the curve. The overall implementation consumes only 3.14 MB of memory and takes 3.96 s of execution time on "Cortex-A53 (ARMv8) 64-bit SoC @1.4 GHz and 1-GB LPDDR2 SDRAM" device with a *Raspberry Pi OS*. The total execution time for the proposed AKA was only 0.1 s on the Linux workstation.

V. CONCLUSION

The research work presented in this letter details a novel lightweight PUF-based protocol for AKA, which is well-suited to establish secure communication between two IoT devices. The key idea is to use a FE to generate random private keys, which enables storage of only a limited amount of public information at the authentication server, thereby enhancing the security and reliability of the PUFs involved and also making the scheme scalable to a large number of IoT devices. The scheme is resilient against both—active and passive—adversarial attacks and was formally verified using the symbolic formal verification tool, *Verifpal*. We also implemented a prototype to demonstrate the proposed scheme practically. Our future task would be to extend our scheme for AKA without the help of verifiers (servers) and consider its applicability in vehicular applications, blockchains, and smartphones.

REFERENCES

[1] R. S. Pappu, "Physical one-way functions," Ph.D. dissertation, Program Media Arts Sci., Massachusetts Inst. of Technol., Cambridge, MA, USA, 2001.
[2] S. Li, T. Zhang, B. Yu, and K. He, "A provably secure and practical PUF-based end-to-end mutual authentication and key exchange protocol for IoT," *IEEE Sensors J.*, vol. 21, no. 4, pp. 5487–5501, Feb. 2021.
[3] A. Braeken, "PUF based authentication protocol for IoT," *Symmetry*, vol. 10, no. 8, p. 352, 2018.
[4] J. Guajardo, S. S. Kumar, G.-J. Schrijen, and P. Tuyls, "FPGA intrinsic PUFs and their use for IP protection," in *Proc. Cryptogr. Hardw. Embed. Syst.*, 2007, pp. 63–80.
[5] F. Zerrouki, S. Ouchani, and H. Bouarfa, "PUF-based mutual authentication and session key establishment protocol for IoT devices," *J. Ambient Intell. Humaniz. Comput.*, vol. 14, pp. 12575–12593, Aug. 2022.
[6] M. Barbosa et al., "SoK: Computer-aided cryptography," in *Proc. 2021 IEEE Symp. Secur. Privacy (SP)*, 2021, pp. 777–795.
[7] N. Kobeissi, G. Nicolas, and M. Tiwari, "Verifpal: Cryptographic protocol analysis for the real world," in *Proc. Int. Conf. Cryptol. India*, 2020, pp. 151–202.
[8] C. Herder, M.-D. Yu, F. Koushanfar, and S. Devadas, "Physical unclonable functions and applications: A tutorial," *Proc. IEEE*, vol. 102, no. 8, pp. 1126–1141, Aug. 2014.
[9] "SoC security." Intel Corporation. [Online]. Available: https://www.intel.com/content/www/us/en/docs/programmable/683711/21-2/soc-security.html (Accessed: Mar. 2023).
[10] W. Buchanan. "ECDH with secp256k1 using Python." Accessed: Dec. 2022. [Online]. Available: https://asecuritysite.com/ecdh/python_secp256k1ecdh
[11] C. Yagemann. "Fuzzy extractor." Accessed: Jun. 2023. [Online]. Available: https://github.com/carter-yagemann/python-fuzzy-extractor
[12] C. Bösch, J. Guajardo, A. Sadeghi, J. Shokrollahi, and P. Tuyls, "Efficient helper data key extractor on FPGAs," in *Proc. Cryptogr. Hardw. Embed. Syst.–CHES 2008*, 2008, pp. 181–197.